

GreatLight PROJECT

SANCTUARY

GAME RULEBOOK & SOFTWARE PROJECT SPECIFICATION

Version 3.4

Professional layout • Complete CLI error handling • Browser-ready architecture

This edition preserves the current rules and architecture, retains the improved table wrapping from v3.3, and restores the complete CLI error explanation and error-message contract.

1. Document Scope

This specification covers the board, pieces, movement, attacks, status effects, victory conditions, command syntax, CLI errors, software architecture, repository layout, dependencies, installation, development workflow, testing, and implementation roadmap.

2. Core Game Rules

2.1 Board

- Board size: **9 × 9**.
- Files: **a–i**; ranks: **1–9**.
- Team A begins on rank 1; Team B mirrors the formation on rank 9.
- Suggested starting formation on c–g: Soldier, Police, Doctor, Priest, Child.
- One red Substance tile is placed at **e5**.

2.2 Victory and Game Over

- Eliminating/capturing the opposing Child wins the game.
- If a Child becomes **Sick**, the game ends immediately and the opposing team wins.
- If a Child is removed, the game ends immediately in favor of the opposing team.

2.3 Statuses

Status	Rule
Healthy	Normal movement and role abilities.
Sick	Created by a successful Police attack. The target has 3 of its own moves to be cured; otherwise it dies after its third move.
Affected	Created when a non-Priest ends its move on e5. Maximum one-square movement; special role movement/attack is disabled for 2 of that piece's own moves.

3. Pieces, Commands and Abilities

Piece	Movement	Command	Attack / Special
Doctor	1–9 squares, horizontal or vertical, in one straight direction.	DU4 / DD2 / DL7 / DR1	Cures adjacent allied Sick/Affected pieces. If shot by Police, is removed immediately.
Priest	Chess-knight L shape; first letter = 1-step leg, second = 2-step leg.	PLU / PUL / PRD / PDR	No attack. Immune to Substance. Can block Police fire; does not block Soldier fire.
Child	Exactly 1 square U/L/R/D.	CD	No attack. Becoming Sick ends the game immediately.
Police	Exactly 2 squares U/L/R/D.	POD	POK . Successful hit causes Sick; Priest can block the shot.
Soldier	Exactly 1 square U/L/R/D.	SD	SK . Unlimited straight-line range; no piece blocks fire; successful hit immediately removes target.

3.1 Doctor

The Doctor moves 1–9 squares horizontally or vertically in one straight direction. Distance is mandatory and must keep the destination on the board. Examples: DU4, DD2, DL7, DR1.

The Doctor cures an adjacent allied Sick or Affected piece. A Police attack against the Doctor is a special case: the Doctor goes out immediately rather than entering the normal Sick sequence.

3.2 Priest

The Priest moves like a chess knight. The command identifies the 1-step leg first and the 2-step leg second. Examples: PLU = 1 left + 2 up; PUL = 2 up + 1 left; PRD = 1 right + 2 down; PDR = 2 down + 1 right. L/R cannot both occur, and U/D

cannot both occur. The Priest may jump over pieces.

The Priest is immune to Substance and can block a Police shot when it lies between the Police and its intended target. It does not block Soldier fire.

3.3 Child

The Child moves exactly one square in a cardinal direction using CD . It cannot attack. If a Child becomes Sick, the game ends immediately and the other team wins.

3.4 Police

Police moves exactly two squares in one cardinal direction using POD . Its attack command is POK . A successful Police attack normally makes the target Sick.

Sickness belongs to the target's own move counter: SICK(3) → target's own move → SICK(2) → target's own move → SICK(1) → target's own move → SICK(0) → death if not cured. A Doctor can cure the target before the deadline.

Police fire can be blocked by a Priest positioned between Police and the intended target. Police shooting a Doctor removes the Doctor immediately. Police shooting a Child makes the Child Sick and therefore ends the game immediately.

3.5 Soldier

Soldier moves exactly one square in a cardinal direction using SD . Its attack command is SK . Soldier fire travels any distance in a straight line, and **no piece blocks it**. A successful hit immediately removes the target. Soldier can therefore shoot through a Priest. Killing a Child ends the game immediately.

3.6 Substance

There is one red Substance tile at e5. Every role except Priest can be affected. A non-Priest that ends its move on e5 becomes Affected.

While Affected, the piece is limited to one-square movement and its special role movement/attack is disabled. The condition lasts for two of that piece's own moves, then returns Healthy. The Doctor can cure an adjacent Affected ally.

4. Command-Line Contract

INPUT

Read one command for the current player.

VALIDATE

Parse syntax.
Check ownership and turn.
Check board bounds.
Check movement/attack legality.
Check status restrictions.
Check line-of-fire / blocking rules.

APPLY

Only after validation succeeds:
update position / status / removals
update sickness or substance counters
evaluate victory
switch turn when appropriate

OUTPUT

Render authoritative state.
Report errors without changing state.

Invalid commands must have **no game-state side effects**. The same engine must be callable from the CLI and later from a browser interface.

5. CLI Error Handling and Error Explanation

The CLI must distinguish between a **validation error** and a **game-state outcome**. Validation errors reject the command and leave the state unchanged. Game-state outcomes are produced only after a valid command has been applied.

5.1 Error Categories

Category	Meaning	Example
SYNTAX	The command cannot be parsed according to the command grammar.	ERROR: INVALID SYNTAX — expected SD .
TURN	The command refers to a piece that does not belong to the current player.	ERROR: WRONG PLAYER — this is not your piece.
BOUNDS	The requested destination is outside the 9×9 board.	ERROR: OUT OF BOUNDS — destination must be on a–i / 1–9.
MOVEMENT	The piece exists, but the requested movement is illegal for that role.	ERROR: INVALID MOVE — Police must move exactly 2 squares.
STATUS	The piece's current status prevents the requested action.	ERROR: STATUS RESTRICTION — Affected pieces may move only 1 square.
ATTACK	The attack command is invalid for the selected role, direction, or target.	ERROR: INVALID ATTACK — Priest has no attack.
BLOCKED	A rule-specific blocker prevents the attack.	ERROR: BLOCKED ATTACK — Police fire is blocked by a Priest.
GAME_OVER	The game has already ended; no further commands are accepted.	GAME OVER: Team A wins.

5.2 Required Error Behavior

- **Reject the command.** Do not apply any part of an invalid command.
- **Keep the state unchanged.** Position, status, counters, turn, and victory state must remain exactly as before the invalid command.
- **Explain the reason.** The message should tell the player what rule was violated.
- **Do not produce a misleading success message.** A rejected command must never be reported as a move or attack.
- **Do not switch turns.** The current player retains the turn after an invalid command.
- **Game-over is final.** Once a victory condition is reached, later commands are rejected.

5.3 Error Message Examples

ERROR: INVALID SYNTAX
Expected: SD <direction>
Example: SD U

ERROR: INVALID MOVE
Doctor cannot move 10 squares.
Maximum Doctor movement distance is 9.

ERROR: OUT OF BOUNDS
Destination i10 is outside the 9×9 board.

ERROR: WRONG PLAYER
You may only control your own pieces.

ERROR: INVALID MOVE
Police must move exactly 2 squares in one cardinal direction.

ERROR: STATUS RESTRICTION
This piece is Affected and may move only 1 square.
Its special movement/attack is disabled.

ERROR: INVALID ATTACK
Priest has no attack.

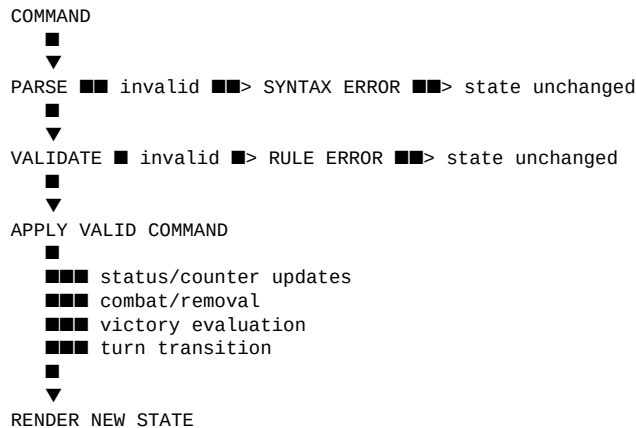
ERROR: BLOCKED ATTACK
Police fire is blocked by a Priest.

ERROR: INVALID TARGET
No valid opposing piece is present in the selected attack direction.

GAME OVER: CHILD SICK
Team B wins because Team A's Child became Sick.

The exact wording may evolve during implementation, but the **semantic contract** must remain stable: category, reason, and absence of state mutation.

5.4 Validation Pipeline



6. Recommended Repository Structure

```
sanctuary/
  README.md
  LICENSE
  .gitignore
  pyproject.toml
  requirements.txt
  docs/
    SANCTUARY_RULEBOOK.pdf
  src/
    sanctuary/
      __init__.py
      domain/
        board.py
        pieces.py
        positions.py
        states.py
        game.py
      rules/
        movement.py
        combat.py
        status.py
        victory.py
        validator.py
      cli/
        parser.py
        commands.py
        renderer.py
      web/
        server.py
        renderer.py
        templates/
          board.html
        static/
          css/
            sanctuary.css
          js/
            sanctuary.js
    main.py
  tests/
    validation/
    state/
    rendering/
    web/
    fixtures/
  tools/
```

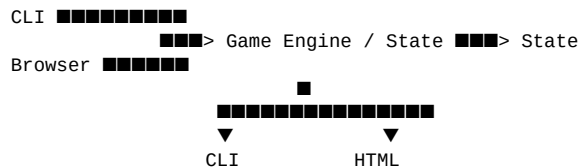
6.1 What Each Area Owns

Path	Responsibility
src/sanctuary/domain/	Authoritative game objects and state: board, positions, pieces, statuses, turns, and game state.
src/sanctuary/rules/	Pure game rules: movement, combat, status transitions, victory, and validation.
src/sanctuary/cli/	Command parsing, command handling, error reporting, and ASCII/text rendering.

Path	Responsibility
src/sanctuary/web/	Browser server, HTML rendering, CSS, and later JavaScript interaction.
tests/	Automated validation of rules, state transitions, rendering, and browser-facing behavior.
docs/	Human-readable specification and project documentation.
tools/	Developer utilities such as fixture generation or local helpers; not game rules.

6.2 Architectural Boundary

Game engine must not depend on CLI or browser code. A browser request should call the same rule/state engine that a CLI command calls.



7. Dependencies and Required Software

Dependency	Required when	Purpose
Python 3.12+	All phases	Primary implementation language for the engine and CLI.
Git	All phases	Version control and collaboration.
pytest	Phases 1–6	Automated unit and integration tests.
Flask	Phase 4+	Small local Python web server for browser rendering and API-style requests.
HTML5 / CSS3	Phase 4+	Browser board and presentation.
JavaScript	Phase 5+	Interactive browser controls; not required for the first HTML renderer.
Web browser	Phase 4+	View and test the generated browser board.

Java is not required for the initial implementation. Add another language or framework only if a concrete project requirement justifies it.

7.1 Python Packages

```
# requirements.txt
pytest
Flask
```

For the earliest command-line phases, pytest is the only third-party Python package needed. Flask should be added when Phase 4 browser rendering begins. The game engine itself should prefer the Python standard library.

7.2 Development Environment

- Python 3.12 or newer installed and available as **python** / **python3**.
- Git installed and configured.
- A code editor/IDE.
- A modern web browser for Phase 4 onward.
- A terminal/command prompt for CLI development and test execution.

8. Installation / Setup

```
# 1. Clone the repository
git clone <repository-url>
cd sanctuary

# 2. Create a virtual environment
python -m venv .venv

# 3. Activate it
```

```
# Windows:
.venv\Scripts\activate
# macOS / Linux:
source .venv/bin/activate

# 4. Install dependencies
python -m pip install --upgrade pip
pip install -r requirements.txt

# 5. Run tests
pytest

# 6. Run the CLI (once implemented)
python -m sanctuary.main
```

A virtual environment keeps project dependencies isolated from the system Python installation. Do not commit `.venv/` to Git.

8.1 Recommended pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "sanctuary"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = []

[project.optional-dependencies]
dev = ["pytest"]
web = ["Flask"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

9. Development Workflow

- 1. Define or confirm the rule.** The rulebook is the source of truth for game behavior.
- 2. Write or extend a test.** Every non-trivial rule should have a positive and negative case.
- 3. Implement the smallest engine change.** Keep rule logic out of renderers.
- 4. Run the full test suite.** Do not rely only on the test currently being written.
- 5. Render the state.** CLI first; HTML later.
- 6. Commit a coherent change.** Keep commits focused and descriptive.

9.1 Suggested Git Commit Style

```
feat: add police sickness countdown
fix: prevent priest from blocking soldier fire
test: cover doctor removal by police
refactor: separate movement validation from state mutation
docs: update sanctuary rulebook
```

10. Testing Strategy

Testing is divided by project phase so that the rules can be validated before browser work begins.

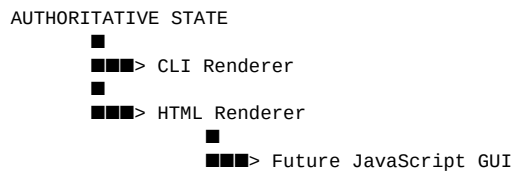
Phase	Test focus
1 — Validation	Command syntax, bounds, ownership, legal/illegal movement, attack legality, error categories, and no mutation on invalid commands.
2 — State	Movement application, combat, sickness countdown, Doctor cure, Soldier removal, Substance, turn switching, and victory.
3 — Rendering	Stable board coordinates, role/team/status display, current-player display, and readable errors.
4 — GUI	HTML generation, browser routes, CSS board structure, and parity with engine state.

10.1 Mandatory Edge Cases

- Doctor distance 1 and 9; out-of-bounds distance.
- Priest's two equivalent L-order encodings and illegal same-axis/opposite-axis combinations.
- Police exactly two squares; blocked Police shot by Priest.
- Soldier long-distance shot through a piece, including through Priest.
- Police → normal target = Sick(3); cure before deadline; uncured death after the target's third own move.
- Police → Doctor = immediate removal.
- Police → Child = Sick → immediate game over.
- Soldier → Child = immediate removal → game over.
- Substance affects non-Priest but not Priest; Affected movement restriction and recovery.
- Invalid command leaves the complete state unchanged.

11. Rendering and Browser Architecture

Do **not** save a screenshot/PNG after every move as the primary game representation. Save the structured game state. Render that state whenever a CLI or browser view is needed.



During Phase 4, regenerate the HTML representation after each valid move and refresh the local browser. In Phase 5, JavaScript can make the board interactive while continuing to use the same underlying rules/state model.

Screenshots may still be generated optionally for debugging, documentation, visual regression, or replay archives. They are never the authoritative state.

12. Implementation Roadmap

Phase	Deliverable
1. Rules & Validation	Parser, command model, legality checks, ownership, bounds, status restrictions, line-of-fire, error contract, and zero side effects on invalid input.
2. Game State Engine	State transitions, movement, combat, Police sickness, Soldier death, Doctor-shot special case, Child game-over, Substance, recovery, turns, victory.
3. CLI Renderer	Readable ASCII board, coordinates, role/team/status, current player, prompt, and professional error output.
4. HTML Renderer	HTML/CSS board generated from authoritative state; local Flask server and browser refresh.
5. Interactive Web GUI	JavaScript selection, legal move feedback, attack selection, and interaction while reusing the Python engine.
6. Full Application	Persistence, replay, animation, accessibility, multiplayer, deployment, and other polish.

13. Implementation Checklist

- Create the repository and Python package layout.
- Add pyproject.toml, requirements.txt, .gitignore, and README.md.
- Implement immutable/value-like position and direction representations.
- Implement piece and status models.
- Implement board occupancy and lookup.
- Implement command parser and validation.
- Implement structured CLI error categories and messages.
- Implement movement rules for all pieces.
- Implement Police and Soldier combat with their distinct blocking/death rules.

- Implement Sick(3) own-move countdown and Doctor cure.
- Implement Substance/Affected two-own-move lifecycle and Priest immunity.
- Implement immediate Child game-over conditions.
- Implement CLI rendering.
- Add comprehensive pytest coverage, including no-mutation-on-error tests.
- Add Flask/HTML renderer only after the engine is stable.
- Keep browser code dependent on the engine, never the reverse.

14. Project Principle

SANCTUARY should be built as a **game engine first, interface second**. The engine owns truth. The CLI and browser are views/controllers around that truth. This keeps the implementation testable now and makes the later HTML/CSS/JavaScript interface an extension rather than a rewrite.

PROJECT PARTS

This section defines how the SANCTUARY project is divided into implementation parts. The Parts section is the project execution specification: each part establishes a bounded set of behaviour that must be specified and tested before the project proceeds to the next part. The parts build on the same authoritative game-state and game-engine architecture defined earlier in this specification.

The Parts section describes **what the program must do**. It is not an implementation guide and does not prescribe source-code solutions. The corresponding acceptance tests define the observable input and complete command-line output for each scenario.

Part 1 — Program Startup and Basic Game Lifecycle

Part 1 establishes the program's first complete lifecycle: receiving startup arguments, validating them, creating a new game, presenting the initial game, accepting the basic quit command, and terminating with the appropriate result. It also establishes the foundation for the later Child-death win condition and repeated-state draw rule.

Part 1 phases

Phase	Purpose
1	Program startup and argument validation
2	Player and initial game definition
3	Initial game presentation
4	Basic player turn and quit
5	Game-end and repetition foundation

1. Program startup arguments

The program accepts three startup arguments: **<game type> <player A name> <player B name>**. Game type 0 means NoGUI/CLI. Game type 1 means GUI/Website.

2. Startup validation order

Validation occurs in this exact order: (1) game type, (2) Player A name, (3) Player B name, (4) comparison of the two names. If a validation step fails, the required error is printed and the program terminates. No welcome message or game session is started after failed validation.

3. Game type validation

Only 0 and 1 are valid game-type values. Any other value produces:

```
ERROR: Game type number should be either 0 or 1, got <gotten argument>
```

4. Player-name validation

A player name must not be an integer and must not consist only of characters. Under the Part 1 rule, a valid name contains at least one character and at least one integer/digit. Examples of valid names include Name1, Player7, User2026 and A1. Examples of invalid names include 123 and Name. An invalid name produces:

```
ERROR: Invalid player name
```

5. Name comparison

Player names are case-sensitive. Name1 and name1 are different. If both names individually pass validation and are identical, the program prints:

```
ERROR: Player names should not be the same
```

6. Successful game startup

Only after all startup validation succeeds is a new game created. Player A is the first player. The program prints the following initial session information:

```
Welcome to a new game:
Player A: <Player A name>
Player B: <Player B name>
Player A turn -> : PLAY
```

The initial board is presented as part of the successful game presentation. Its exact canonical representation remains governed by the board/rendering specification.

7. Complete CLI scenario

Part 1 acceptance tests are complete command-line scenarios. Each input file contains all input supplied to the program for that scenario. Each expected-output file contains the complete output printed from program start until the program terminates. Expected-output files are therefore complete transcripts, not isolated fragments.

8. Quit command

For Part 1, q is the quit command for the current player. A quitting player immediately loses and the other player wins. The required Player A quit result is:

```
Player A -> Quits
WIN GAME: Player B wins
```

The corresponding Player B result is Player B -> Quits followed by WIN GAME: Player A wins. The program terminates after the win result.

9. Part 1 canonical complete scenario

Input:

```
0 Name Name1
q
```

Complete expected output:

```
Welcome to a new game:
Player A: Name
Player B: Name1
Player A turn -> : PLAY
Player A -> Quits
WIN GAME: Player B wins
```

10. Terminal conditions

The project defines terminal win conditions for a player quitting and for the opposing Child dying. Part 1 establishes the quit condition; the Child-death condition will become active when the corresponding game rules are implemented.

A draw occurs when the same complete game state occurs five times with the same player-to-move. The required result is:

```
DRAW PLAY: Same game state 5 times.
```

11. Part 1 implementation/acceptance requirements

1. Validate game type before player names.
2. Accept only game types 0 and 1.
3. Validate Player A before Player B.
4. Reject invalid individual player names.
5. Compare names only after both names individually pass validation.
6. Treat names as case-sensitive.
7. Reject identical player names.
8. Do not start a game after failed startup validation.

9. Do not print the welcome message after failed startup validation.
10. Start Player A first after successful validation.
11. Print the complete successful-startup presentation.
12. Recognize q as the current player's quit command.
13. Award the win to the other player when the current player quits.
14. Terminate after a quit win.
15. Maintain the state information required for future Child-death and repetition rules.
16. Include the current player in repetition-state identity.
17. Declare a draw on the fifth occurrence of the same complete game state with the same player-to-move.
18. Use complete CLI transcripts as the acceptance-test model.

Part 1 Specification — Correction

Player-name comparison is case-sensitive.

Therefore, Name1 and name1 are different player names.

They are valid as two different names, provided both individually satisfy the player-name validity rules.

Only the exact same spelling and capitalization is considered the same.

Example:

```
Name1 == Name1  -> same
```

```
Name1 == name1  -> different
```

Part 1 test case correction

The case-sensitive-name test must use Name1 and name1 and must expect successful startup, because the names are different.

Part 1 Specification — Name Comparison Correction

Player-name comparison is NOT case-sensitive.

Names that differ only by capitalization are considered the same name.

Examples:

Name1 and Name1 -> same

Name1 and name1 -> same

NAME1 and name1 -> same

If Player A and Player B have the same name when capitalization is ignored, the program must print:

ERROR: Player names should not be the same

The Part 1 case test therefore uses:

0 Name1 name1

and expects the same-name error.